

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0603

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 15

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Mohammed Mudhassir A/192424367(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Mudhassir

1. Data Preprocessing and Cleaning

Input: Raw dataset file

Output: Cleaned dataset file

ALGORITHM Preprocess_Dataset

INPUT: Dataset file

OUTPUT: Cleaned dataset

1. START
2. Read dataset from file into DataFrame DF.
3. Check number of rows, columns, data types, and missing values.
4. For each numerical column:
 - a. Identify missing values.
 - b. Replace missing values with column median.
5. For each categorical column:
 - a. Remove leading/trailing spaces.
 - b. Convert values to a common case.
 - c. Replace missing values with mode or "Unknown".
6. Remove duplicate records.
7. Filter invalid numerical entries using valid ranges.
8. Normalize numerical columns:
$$\text{Normalized} = (\text{Value} - \text{Minimum}) / (\text{Maximum} - \text{Minimum})$$
9. Reset the DataFrame index.
10. Validate that:
 - a. Required columns exist.
 - b. Missing values are handled.
 - c. Duplicate records are removed.
 - d. Numerical values are valid.
11. Save cleaned DataFrame to a new file.
12. STOP

Efficiency: Use vectorized array/DataFrame operations instead of row-by-row loops.

2. Statistical Measures from a Large Dataset

Input: Numerical dataset

Output: Mean, median, standard deviation, and variance

ALGORITHM Calculate_Statistics

INPUT: Numerical dataset

OUTPUT: Statistical measures

1. START
2. Load dataset into an array.
3. Check whether the array is empty.
4. If empty:
 - Display "No data available"
 - STOP
5. Remove or filter missing, invalid, or unwanted values.
6. Identify extreme outliers using an appropriate threshold.
7. Calculate:
 - Mean = $\text{SUM}(\text{values}) / \text{COUNT}(\text{values})$
 - Median = middle value after sorting
 - Variance = $\text{Average}((\text{value} - \text{Mean})^2)$
 - Standard Deviation = $\text{SQRT}(\text{Variance})$
8. Use vectorized aggregation functions for efficiency.
9. Validate results:
 - a. Count of valid observations
 - b. Minimum and maximum values
 - c. Check for NaN or infinite results
10. Compare results with expected ranges.
11. Display the statistical measures.
12. STOP

For very large datasets, process the data in chunks and maintain running aggregates to reduce memory usage.

3. Combining Multiple DataFrames

ALGORITHM Combine_DataFrames

INPUT: DataFrames DF1, DF2, ..., DF_n

OUTPUT: Merged DataFrame

1. START
 2. Load all DataFrames.
 3. Check column names and data types.
 4. Identify the common key.
 5. Align indexes and standardize key formats.
 6. Remove exact duplicate records.
 7. Merge DataFrames using the common key.
 8. Use an outer merge when preservation of all valid records is required.
 9. Group by the common key.
 10. Detect duplicate or conflicting records.
 11. Resolve conflicts using predefined rules.
 12. Sort the merged DataFrame by the key.
 13. Reset the index.
 14. Validate:
 - a. Required records are present.
 - b. No unexpected duplicate keys exist.
 - c. Important columns are not lost.
 - d. Record counts are reasonable.
 15. Generate an integrity report.
 16. Return the final DataFrame.
 17. STOP
-

4. Filtering and Sorting a Dataset

ALGORITHM Filter_And_Sort

INPUT: Dataset DF, filtering conditions

OUTPUT: Filtered and sorted dataset

1. START
 2. Load dataset into DataFrame DF.
 3. Identify required columns.
 4. Define conditions:
 - Condition1 = Value $\geq$ Minimum
 - Condition2 = Value $\leq$ Maximum
 - Condition3 = Category belongs to required categories
 5. Combine conditions using AND/OR logical operators.
 6. Apply Boolean indexing.
 7. Store valid records in Filtered_DF.
 8. Sort Filtered_DF using the required column.
 9. Reset the index.
 10. Validate that every record satisfies all conditions.
 11. For large datasets, select only required columns and avoid unnecessary copies.
 12. Display or save the result.
 13. STOP
-

5. Complete Data Analysis Workflow

ALGORITHM Data_Analysis_Workflow

INPUT: One or more data files

OUTPUT: Summary report and plots

1. START

2. Identify input files.
 3. For each file:
 - a. Try to read the file.
 - b. If an error occurs, record the error and continue.
 4. Convert loaded data into DataFrames.
 5. Inspect columns, data types, missing values, and duplicates.
 6. Clean and transform the data.
 7. Filter invalid records.
 8. Group data based on required categories.
 9. Calculate SUM, MEAN, COUNT, MIN, and MAX.
 10. Sort the summarized results.
 11. Store the summary DataFrame.
 12. Generate suitable plots such as:
 - a. Bar chart
 - b. Line chart
 - c. Histogram
 13. Add titles, labels, and legends.
 14. Save plots and summary results.
 15. Validate the final output.
 16. STOP
-

6. Time-Series Cleaning and Transformation

ALGORITHM Clean_TimeSeries

INPUT: Time-series dataset

OUTPUT: Clean and regularly sampled time-series data

1. START
2. Load the dataset.
3. Convert timestamp column to standard datetime format.

4. Set timestamp as the DataFrame index.
 5. Sort the index chronologically.
 6. Detect missing timestamps.
 7. Create a regular time index at the required frequency.
 8. Reindex the DataFrame using the regular index.
 9. Interpolate missing numerical values using time-based interpolation.
 10. Detect outliers using Z-score or IQR.
 11. Remove or cap extreme outliers.
 12. Filter invalid timestamps and values.
 13. Resample data at the required fixed interval.
 14. Apply aggregation such as mean, sum, or median.
 15. Validate chronological order and missing values.
 16. Return cleaned time-series data.
 17. STOP
-

7. Feature Selection

ALGORITHM Feature_Selection

INPUT: Dataset containing multiple features

OUTPUT: Selected feature dataset

1. START
2. Load the dataset.
3. Separate input features X and target Y.
4. Identify numerical features.
5. Handle missing values.
6. Normalize numerical features.
7. Calculate correlation between every pair of features.
8. Set a correlation threshold.
9. If two features have correlation greater than the threshold:

Keep the more relevant feature.

Remove the redundant feature.

10. Calculate feature relevance with respect to the target.
11. Remove features below the relevance threshold.
12. Store the selected features.
13. Validate the selected feature set.
14. Compare model performance before and after selection.
15. Return the significant features.
16. STOP

This reduces dimensionality, redundancy, and computational cost.

8. Data Partitioning for Machine Learning

ALGORITHM Split_Dataset

INPUT: Dataset DF, target column, split ratios

OUTPUT: Training, validation, and testing datasets

1. START
2. Load dataset.
3. Separate features X and target Y.
4. Remove or handle missing and invalid records.
5. Check class distribution.
6. Randomly shuffle the dataset.
7. If classification is used:
 Apply stratified splitting.
8. Split the data:
 Training = 70%
 Validation = 15%
 Testing = 15%
9. Ensure no record appears in more than one set.

10. Check class distribution in each partition.
 11. Verify that preprocessing parameters are learned only from training data.
 12. Apply the learned transformations to validation and testing data.
 13. Check for data leakage.
 14. Return the three datasets.
 15. STOP
-

9. Outlier Detection Using Z-Score or IQR

ALGORITHM Detect_Outliers

INPUT: Numerical dataset

OUTPUT: Cleaned dataset

1. START
2. Load numerical data.
3. Handle missing values.
4. Select a numerical column.
5. Choose an outlier detection method.
6. For Z-score:
 - Calculate Mean and Standard Deviation.
 - $Z = (\text{Value} - \text{Mean}) / \text{Standard Deviation}.$
 - Mark values where $\text{ABS}(Z) > \text{threshold}$ as outliers.
7. OR use IQR:
 - $Q1 = \text{First Quartile}$
 - $Q3 = \text{Third Quartile}$
 - $\text{IQR} = Q3 - Q1$
 - $\text{Lower} = Q1 - 1.5 \times \text{IQR}$
 - $\text{Upper} = Q3 + 1.5 \times \text{IQR}$

Mark values outside the limits as outliers.

8. Decide whether to:
 - a. Remove outliers, OR
 - b. Cap them at lower/upper limits.
 9. Recalculate summary statistics.
 10. Check that valid observations are preserved.
 11. Compare data before and after cleaning.
 12. Return cleaned dataset.
 13. STOP
-

10. Data Aggregation and Summarization

ALGORITHM Aggregate_Data

INPUT: Dataset DF and grouping attributes

OUTPUT: Summary report

1. START
2. Load dataset.
3. Validate required columns.
4. Handle missing or invalid records.
5. Select grouping attributes.
6. Group records by the specified categories.
7. Calculate:
$$\text{Total} = \text{SUM}(\text{Value})$$
$$\text{Average} = \text{MEAN}(\text{Value})$$
$$\text{Count} = \text{COUNT}(\text{Value})$$
8. Store the aggregated results.
9. Sort the summary by the required metric.
10. Set or reset the index appropriately.

11. Validate that aggregated totals match the original valid data.
 12. Generate a summary report.
 13. Display or save the report.
 14. STOP
-

11. Data Normalization and Scaling

ALGORITHM Normalize_Data

INPUT: Dataset DF

OUTPUT: Scaled dataset

1. START
2. Load dataset.
3. Identify all numerical columns.
4. Handle missing numerical values.
5. Select the required scaling method.

6. For Min-Max Scaling:

Minimum = MIN(column)

Maximum = MAX(column)

Scaled = (Value - Minimum) / (Maximum - Minimum)

7. For Z-score Normalization:

Mean = MEAN(column)

StandardDeviation = STD(column)

Scaled = (Value - Mean) / StandardDeviation

8. Replace the original numerical values with scaled values.
9. Leave categorical columns unchanged.
10. Validate:

Min-Max values should normally lie between 0 and 1.

Z-score data should have approximately mean 0 and standard deviation 1.

11. Check for division by zero in constant columns.

12. Return the scaled dataset.

13. STOP

12. Principal Component Analysis (PCA)

ALGORITHM PCA

INPUT: Numerical dataset X

OUTPUT: Reduced-dimensional dataset

1. START

2. Load dataset X.

3. Handle missing values.

4. Select numerical features.

5. Standardize each feature:

$$X_standard = (X - \text{Mean}) / \text{StandardDeviation}$$

6. Calculate the covariance matrix.

7. Compute eigenvalues and eigenvectors.

8. Pair each eigenvalue with its corresponding eigenvector.

9. Sort eigenvalues in descending order.

10. Select the first K eigenvectors with the largest eigenvalues.

11. Create the projection matrix using selected eigenvectors.

12. Transform the standardized data:

$$\text{Reduced_Data} = X_standard \times \text{Projection_Matrix}$$

13. Validate the new number of dimensions.

14. Calculate explained variance.

15. Confirm that sufficient information is retained.

16. Return Reduced_Data.

17. STOP

Efficient matrix and vectorized operations should be used for large datasets.

13. Data Merging and Conflict Resolution

ALGORITHM Merge_And_Resolve

INPUT: Multiple datasets

OUTPUT: Consistent merged dataset

1. START

2. Load all datasets.
 3. Standardize column names and data types.
 4. Identify the common key.
 5. Set or align indexes using the common key.
 6. Merge the datasets.
 7. Detect missing records.
 8. Detect conflicting attribute values.
 9. For each conflict:
 - a. If one source has higher priority, select its value.
 - b. If numerical values are equally reliable, calculate the average.
 - c. If one value is missing, use the available valid value.
 10. Remove duplicate records.
 11. Validate key uniqueness and schema consistency.
 12. Sort the final dataset.
 13. Reset the index.
 14. Generate a conflict-resolution report.
 15. Return the final dataset.
 16. STOP
-

14. Rolling Window Computations

ALGORITHM Rolling_Statistics

INPUT: Time-series dataset and window size W

OUTPUT: Moving average and rolling variance

1. START
 2. Load the time-series dataset.
 3. Convert timestamps to datetime.
 4. Sort data chronologically.
 5. Set timestamp as the index.
 6. Select the numerical value column.
 7. Define window size W .
 8. For each valid rolling window:
 - a. Extract W consecutive values.
 - b. Ignore or handle missing values according to the selected rule.
 - c. Calculate Mean.
 - d. Calculate Variance.
 9. Store the calculated values.
 10. Assign results to the corresponding timestamps.
 11. For boundary positions where fewer than W observations exist:

Return missing values or use a minimum-period rule.
 12. Validate window alignment.
 13. Use vectorized rolling operations for large datasets.
 14. Return the results.
 15. STOP
-

15. Data Validation and Integrity Checking

ALGORITHM `Validate_Dataset`

INPUT: Raw dataset

OUTPUT: Validated dataset and validation report

1. START
2. Load the dataset.
3. Check whether the file was loaded successfully.
4. Verify required columns are present.

5. Check data types of every column.
6. Detect missing values.
7. Detect duplicate records.
8. Detect invalid or unexpected values.
9. Apply constraints:
 - a. Numerical values must fall within valid ranges.
 - b. IDs must follow the required format.
 - c. Dates must have valid formats.
 - d. Required fields must not be empty.
10. Filter invalid records into an error dataset.
11. Log every detected error with:
 - Record ID
 - Column name
 - Error type
 - Invalid value
12. Correct errors where possible.
13. Remove records that cannot be safely corrected.
14. Revalidate the cleaned dataset.
15. Generate a validation report containing:
 - Total records
 - Valid records
 - Invalid records
 - Missing values
 - Duplicate count
 - Constraint violations
16. If all required checks pass:
 - Mark dataset as "READY FOR ANALYSIS".
17. Otherwise:
 - Mark dataset as "REQUIRES CORRECTION".
18. STOP